Université Abdelmalek Essaâdi
Faculté des Sciences de Tétouan
Département d'Informatique
Filière : Ingénierie Informatique et Développement Logiciel (2IDL)
Année universitaire : 2026- 2027

Module M353 : Analyse et Conception Orientées Objet en Langage UML

Chapitre 2 : Diagramme de Cas d'Utilisation et Diagramme de Package

Pr. Abderrahim EL MHOUTI

a.elmhouti@uae.ac.ma

Plan du chapitre 2 : Diagramme de cas d'utilisation et diagramme de package

- **Diagramme de cas d'utilisation** (use-case diagram)
  - *Introduction*
  - *Eléments d'un diagramme de cas d'utilisation*
  - *Relations dans un diagramme de cas d'utilisation*
  - *Modélisation des besoins avec UML*
- **Diagramme de package** (package diagram)
  - *Notion de package*
  - *Diagramme de packages*
  - *Dépendances entre packages*

Diagramme de cas d'utilisation

Introduction

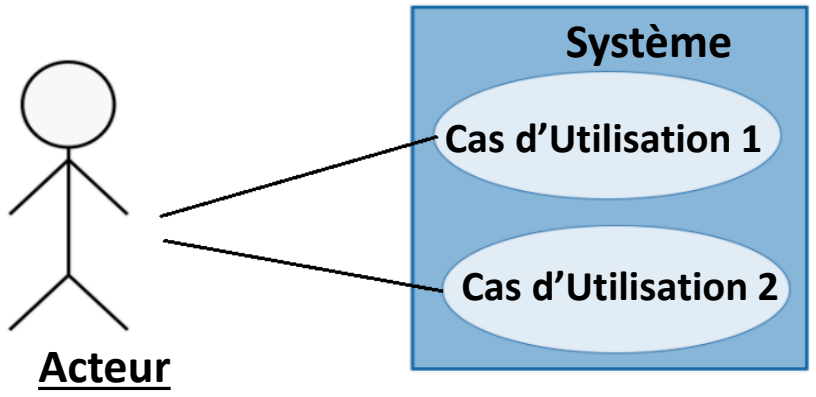
Introduction

Comment modéliser les besoins avec UML?

- Avant de développer un système logiciel, il faut savoir précisément **à quels besoins (QUOI)** devra répondre pour chaque **acteur (QUI)**.

- ➔ Il faut **comprendre**, **clarifier** et **structurer** les attentes et les **besoins** du client en :
  - donnant l'**inventaire des fonctionnalités** et les **intervenants** ;
  - organisant les **besoins** entre eux via des **relations**.

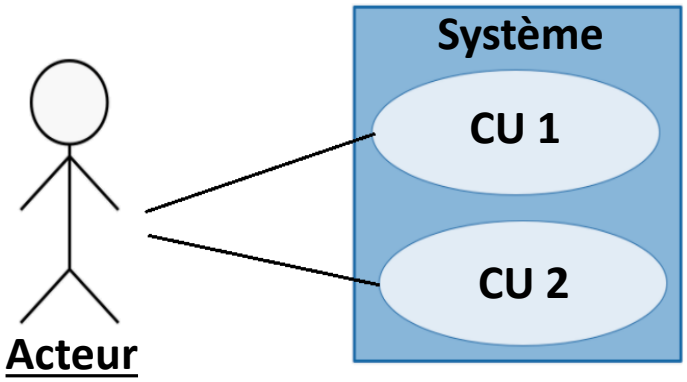
Avec UML, on modélise les besoins au moyen de diagrammes de Cas d'Utilisation (CU).



Introduction

Comment modéliser les besoins avec UML?

- Le **diagramme des Cas d'Utilisation** (CU) constitue la première étape de l'**analyse UML** en permettant de :
  - modéliser les **besoins des utilisateurs**.
  - identifier les grandes **fonctionnalités** et les **limites** du système.
  - représenter les **interactions** entre le **système** et ses **utilisateurs**.



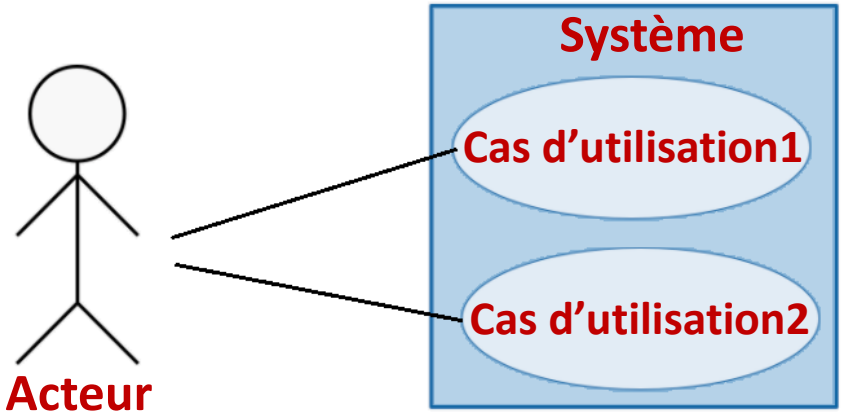
- ***NB*** : Les **cas d'utilisation** (formalisés par **Ivar Jacobson**) apportent une **vision utilisateur** et non pas une **vision informatique**.

Eléments d'un diagramme de cas d'utilisation

Eléments d'un diagramme de cas d'utilisation

3 éléments …

- Le modèle des cas d'utilisation comprend 3 types de composants :
  - **Les acteurs**
  - **Les cas d'utilisation**
  - **Le système**

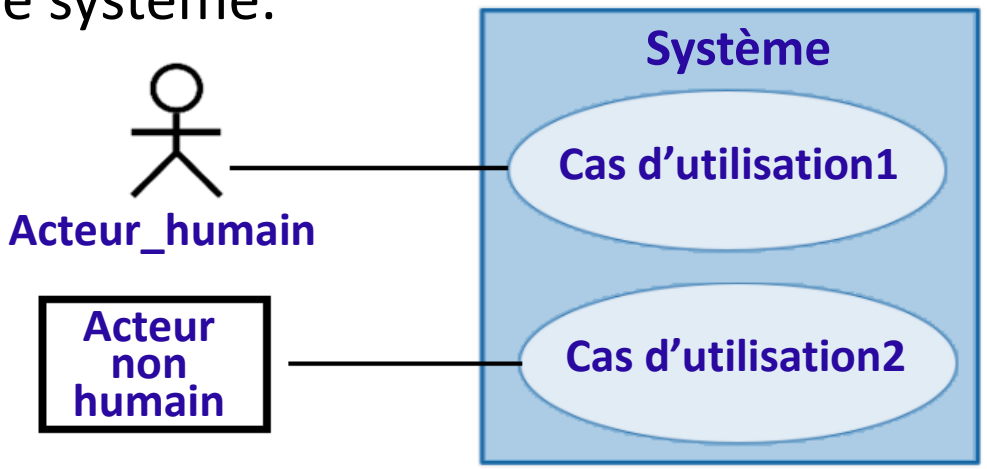


- Les **fonctionnalités** du système (**besoins** des acteurs) sont exprimées sous forme de famille **d'interactions** dans les **cas d'utilisation**.

Eléments d'un diagramme de cas d'utilisation

Acteur

- Un **acteur** :
  - est une **entité** extérieure au système modélisé (*personne, matériel, application, autre système, …*).
  - **interagit** directement avec le système.
  - joue un **rôle** typique.

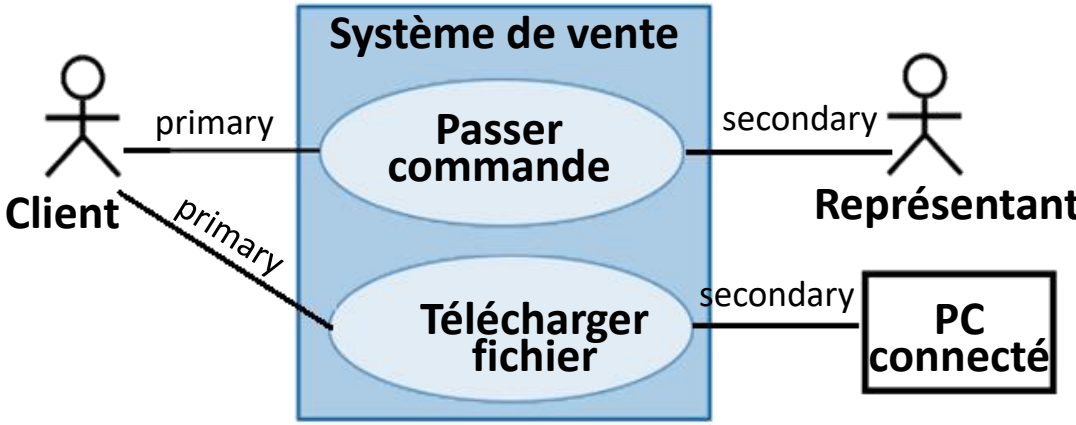


- Graphiquement, les acteurs sont représentés sous la forme d'un :
  - **petit personnage (stick man)** : si l'acteur est humain.
  - **classeur** : si l'acteur est non humain.

Eléments d'un diagramme de cas d'utilisation

Acteur

- Il existe quatre catégories d'acteurs :
  - Les **acteurs principaux**
  - Les **acteurs secondaires**
  - Le **matériel externe**
  - Les **autres systèmes**



- Chaque acteur doit être décrit de manière précise (3 ou 4 lignes).
- ***Exemple*** :



Guichetier

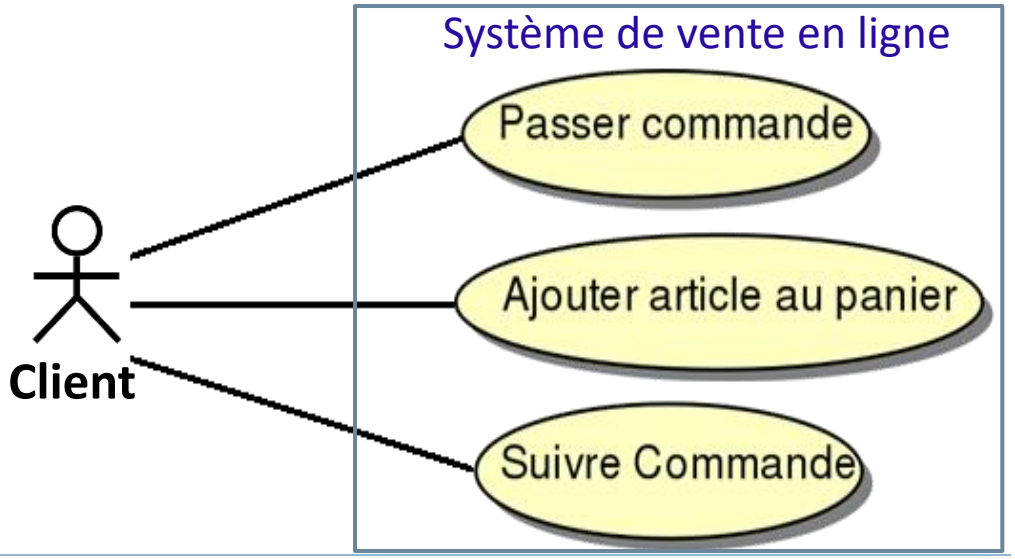
- Un guichetier est un employé de la banque.
- Interface entre les système informatique et les clients.
- Peut effectuer une série d'opération : Création d'un compte, dépôt et retrait d'argent, ..

Eléments d'un diagramme de cas d'utilisation

Cas d'utilisation (CU)

- Un **cas d'utilisation** (CU) décrit une **fonctionnalité** (**service**) du système vue par un « **acteur** » et **utile** à ce dernier.
- C'est un « **ensemble d'événements** » qui se produisent lorsqu'un « **acteur** » utilise un système pour terminer un processus.
- Un **CU** est représenté par un **ovale** relié par des **associations** de type « **participe à** » à ses **acteurs**.
- ***Exemple*** :

Les fonctionnalités d'un système de vente en ligne sont décrites en 3 CU :



Eléments d'un diagramme de cas d'utilisation

Représentation

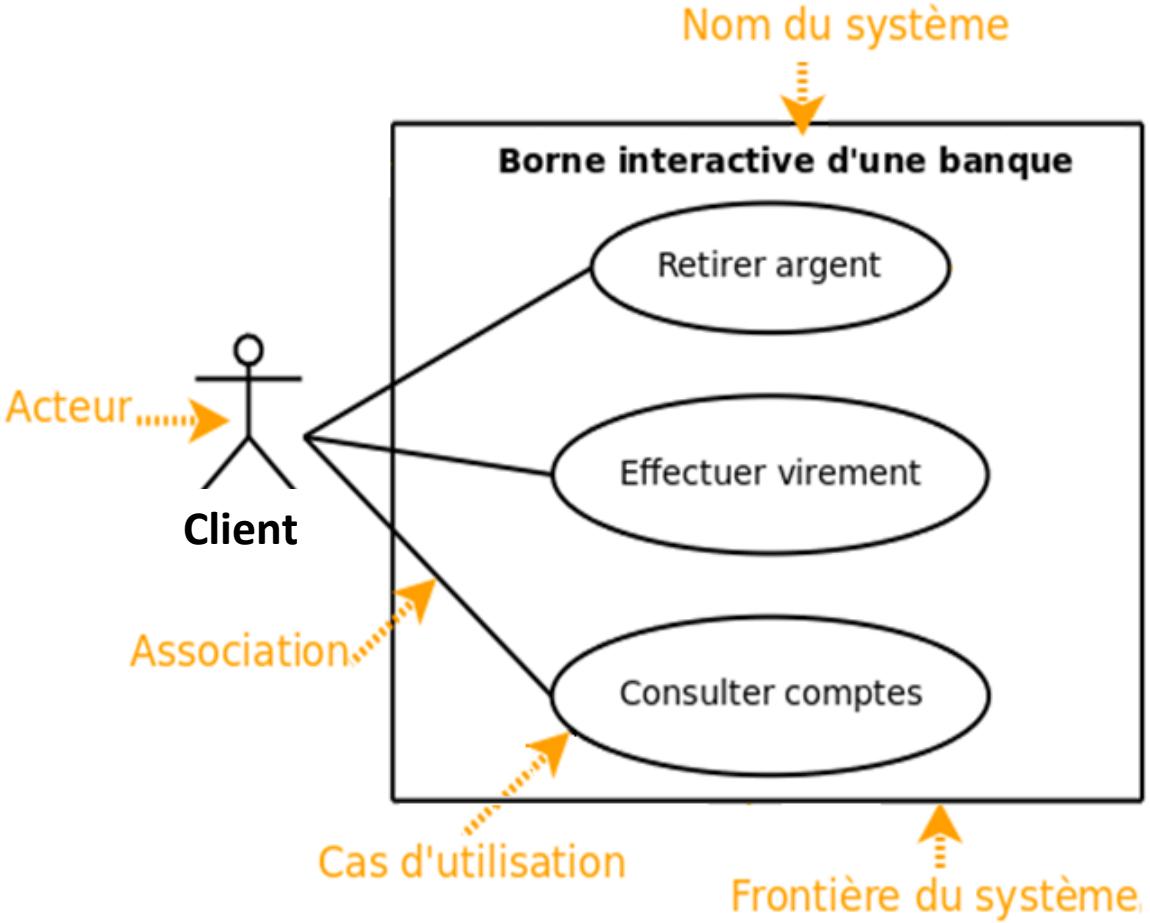
- Exemple simplifié de **diagramme de CU** modélisant une **borne d'accès à une banque**.

Borne banquière



Client

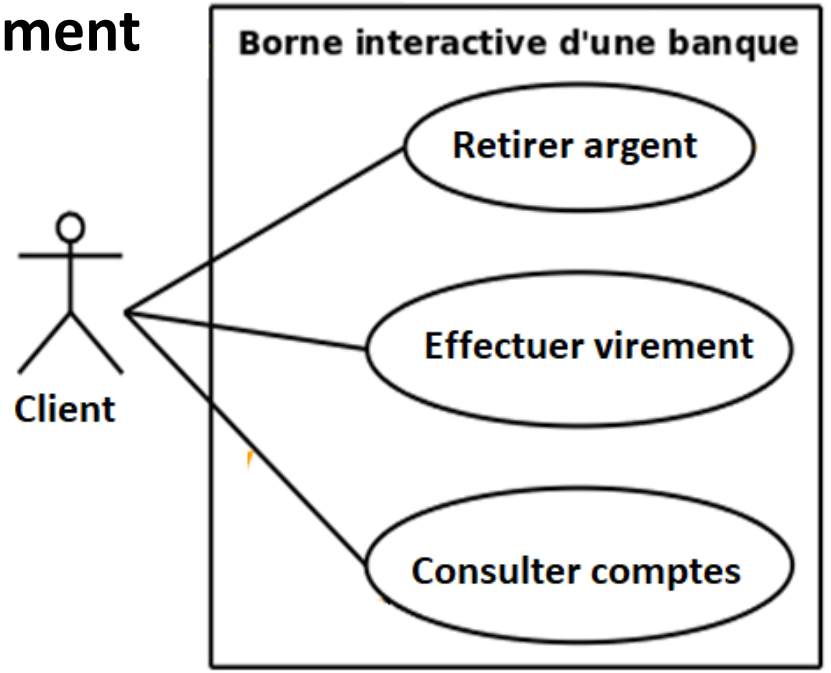
Modélisation UML



Eléments d'un diagramme de cas d'utilisation

Cas d'utilisation (CU)

- L'**ensemble des CU** décrit **exhaustivement** les **fonctionnalités** du système.



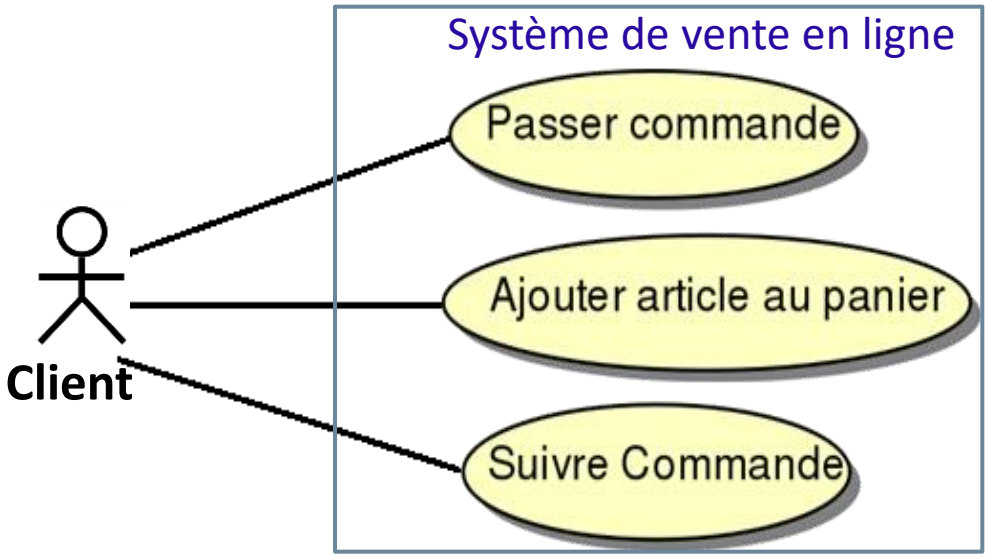
- Pour chaque CU, il faut :
  - choisir un **identificateur** représentatif : **verbe à l'infinitif** + **complément** du point de vue de l'**acteur**.
  - réaliser une **description** détaillée plus au moins formelle : préciser **ce que fait le système**, **ce que fait l'acteur**.

Relations dans un diagramme de CU

Relations dans un diagramme de CU

Relation *Acteur-CU*

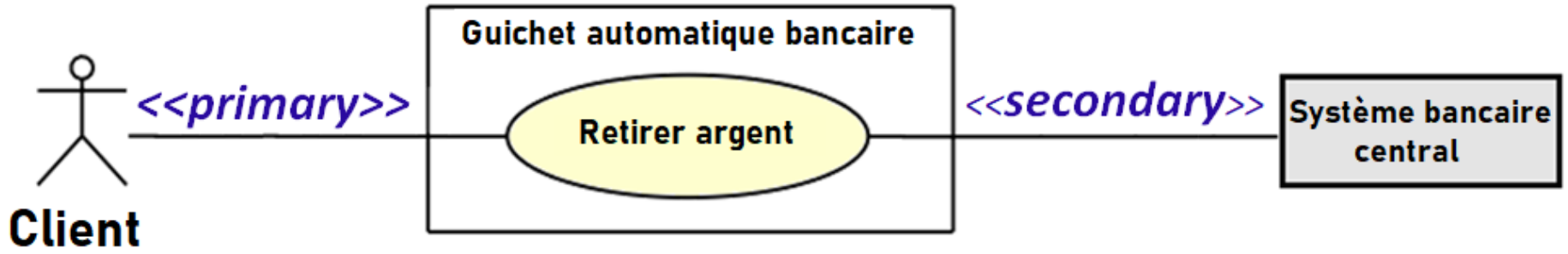
- **Relation d'association** :
  - Les **acteurs** impliqués dans un **CU** lui sont liés par une **association** de type « **participe à** » et représentée par un « **trait continu** ».
  - **Association** = **lien** entre un **acteur** et un **CU** qu'il peut s'exécuter.
  - Un **CU** doit être relié au moins à un **acteur.** Un **acteur** peut utiliser **plusieurs fois** le même **CU**.
  - ***Exemples*** :
- Associations entre le client et trois CU :



Relations dans un diagramme de CU

Relation *Acteur-CU*

- **Acteur principal et secondaire :**
  - Un **acteur** est qualifié de **principal** pour un **CU** lorsque ce **CU** rend service à cet **acteur**.
  - Les **autres acteurs** sont alors qualifiés de **secondaires**.
  - ***Exemple*** : Diagramme de CU d'un GAB (Guichet Auto Bancaire):

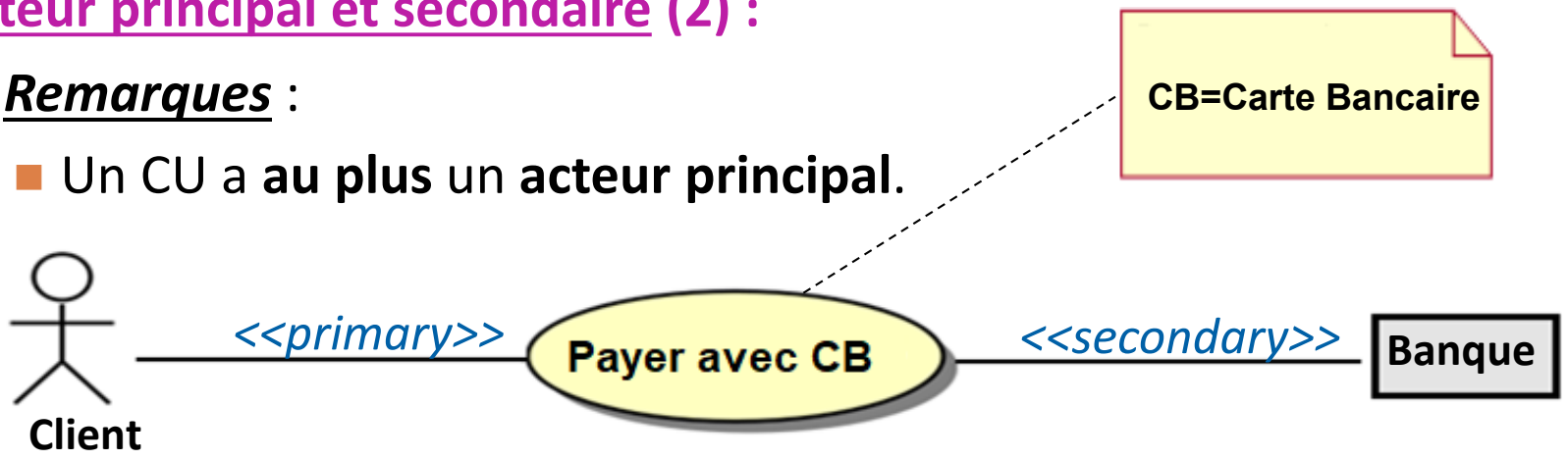


  - Le lien reliant un **CU** à son **acteur principal** est symbolisé par le stéréotype *<<primary>> (<<secondary>> pour l'**acteur secondaire**).*

Relations dans un diagramme de CU

Relation *Acteur-CU*

- **Acteur principal et secondaire (2) :**
  - ***Remarques*** :
    - Un CU a **au plus** un **acteur principal**.

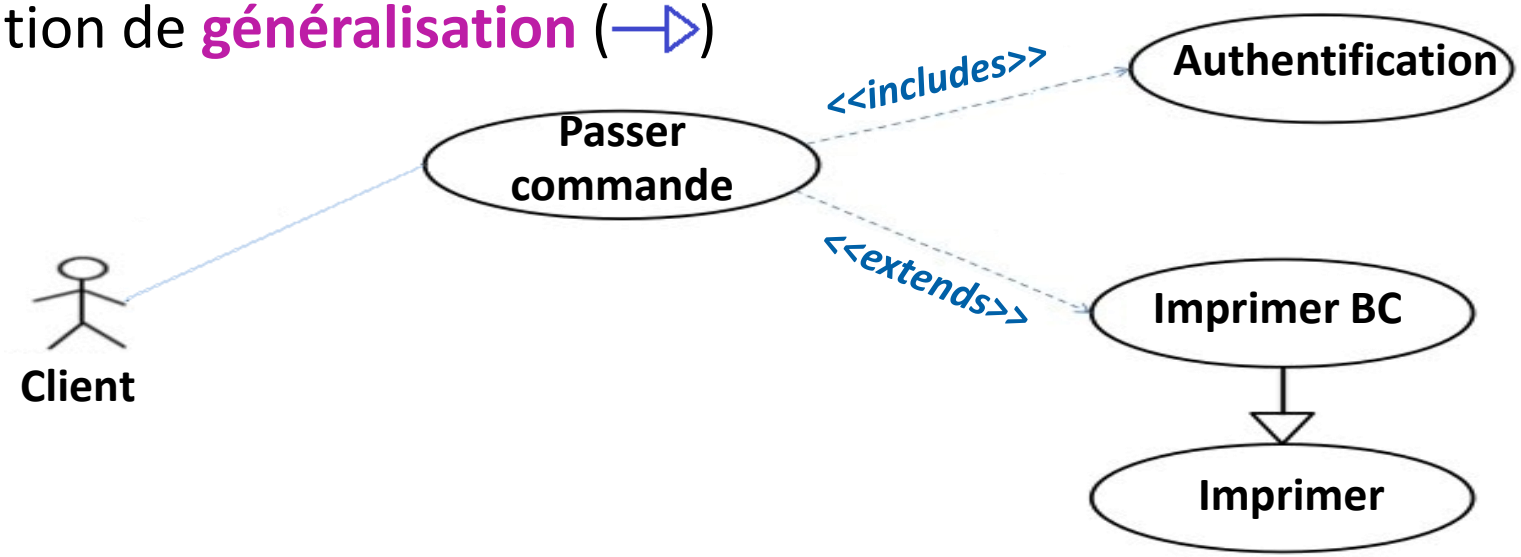


- Pour un CU, un **acteur** est **principal** si c'est l'acteur qui a l'initiative des échanges nécessaires pour réaliser le CU.
- Le plus souvent, les **acteurs secondaires** sont d'autres **systèmes informatiques** avec lesquels le système développé est interconnecté.
- Pour compléter les descriptions, on peut ajouter des **commentaires** en utilisant le symbole avec tu texte à l'intérieur.

Relations dans un diagramme de CU

Relation *CU-CU*

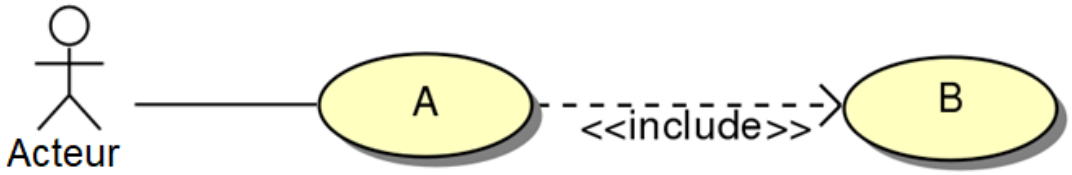
- Les **relations entre CU** permettent la **structuration** des **CU** et la décomposition du système en fonctionnalités à granularité plus fine.
- Il existe 3 types de relations entre CU :
  - relation **d'inclusion** (*<<include>>*)
  - relation **d'extension** (*<<extends>>*)
  - relation de **généralisation** (—▷)



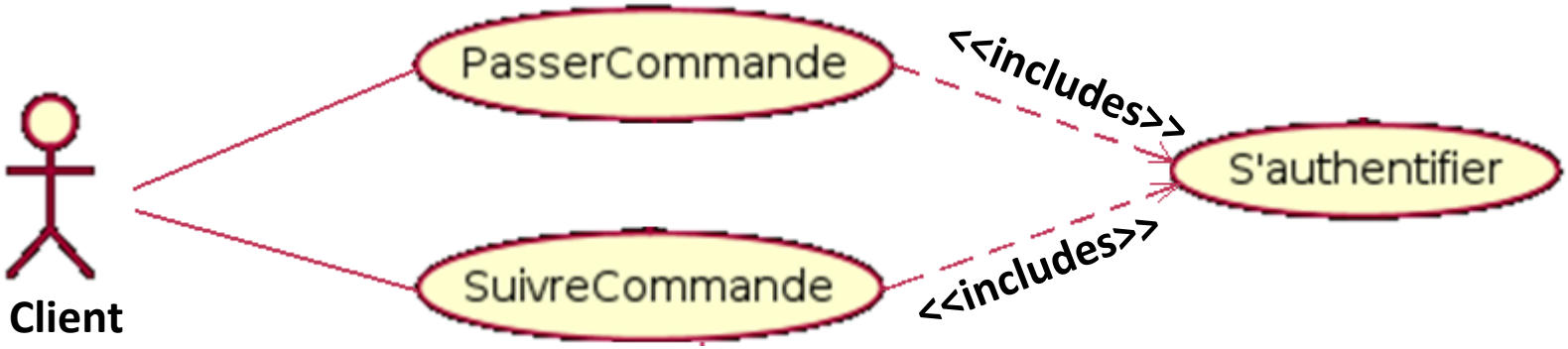
Relations dans un diagramme de CU

Relation *CU-CU :* Inclusion

- On dit que le cas **A inclut** le cas **B,** (dans le sens de la flèche), ssi le cas **B** est une partie **obligatoire** de **A**.
- Cette relation est symbolisée par le stéréotype ***<<include>>***.



- ***Exemple*** : Diagrammes de CU d'un système de vente en ligne :

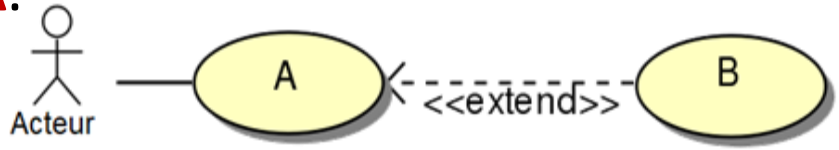


- ***Remarque*** : Les relations entre cas permettent la réutilisabilité des CU.

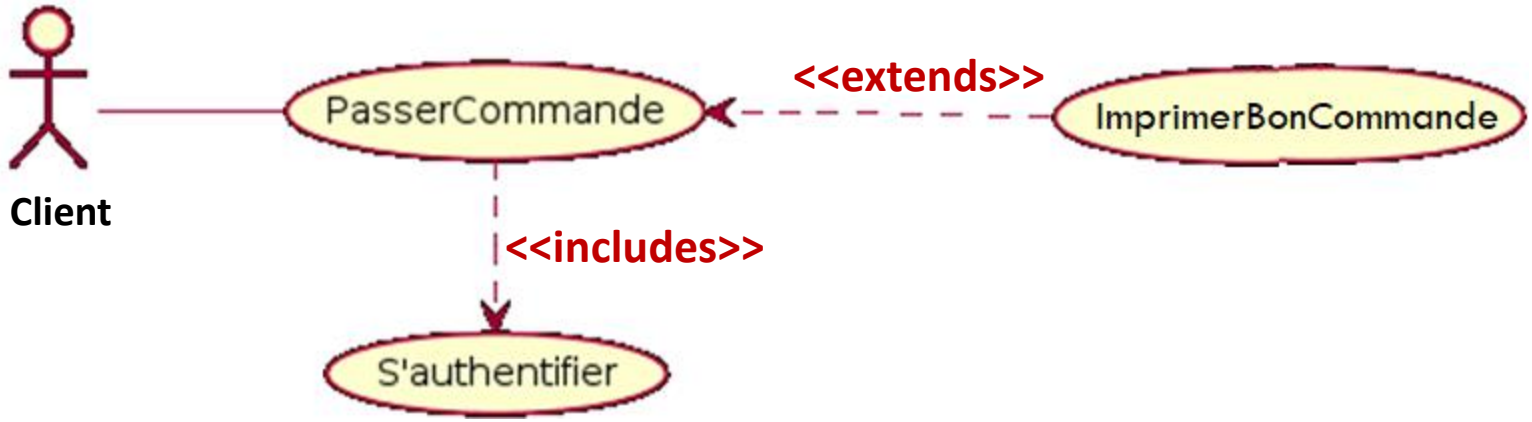
Relations dans un diagramme de CU

Relation *CU-CU :* Extension

- On dit que le cas **B étend** le cas **A** (dans le sens de la flèche), ssi le **cas B** est une partie **optionnelle** de **A**.



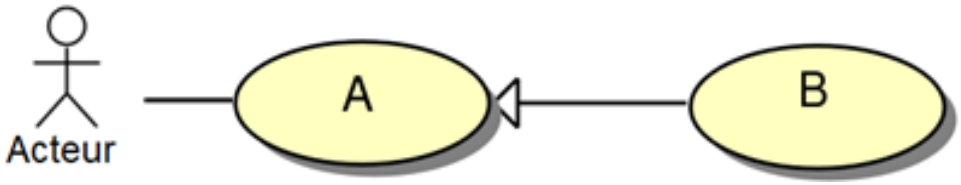
- L'**extension** ajoute une fonctionnalité facultative : c à d : **exécuter A** peut **éventuellement** entraîner l'**exécution de B**.
- Cette relation est symbolisée par le stéréotype ***<< extend >>***
- ***Exemple*** : Diagrammes de CU d'un système de vente en ligne :



Relations dans un diagramme de CU

Relation *CU-CU :* Généralisation

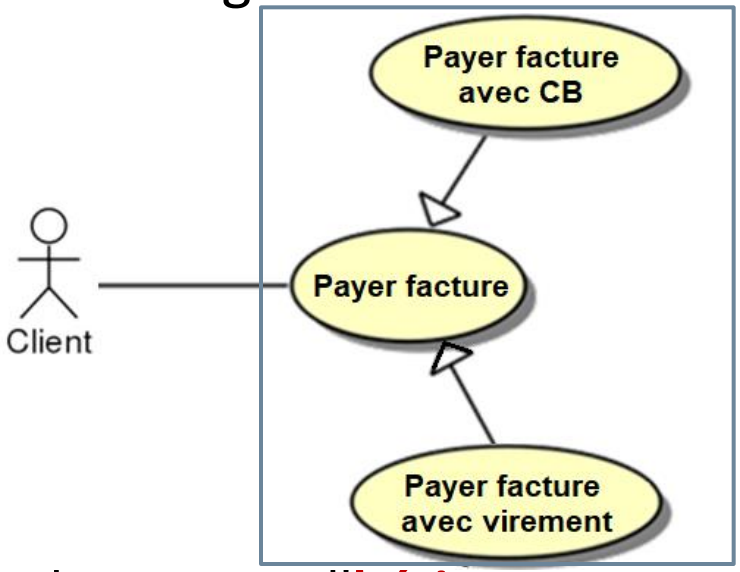
- Un **cas A** est une **généralisation** d'un **cas B** si **B** est un **cas particulier** de **A** (B est une sorte de A).



- La **généralisation** est représentée par une flèche pleine avec triangle en pointe (—▷). La flèche pointe vers l'élément général.

- ***Exemple*** :

Les CUs « *Virement* » et « *Paiement CB* » sont des cas particuliers du CU « *paiement* » *(un virement est une sorte de paiement).*

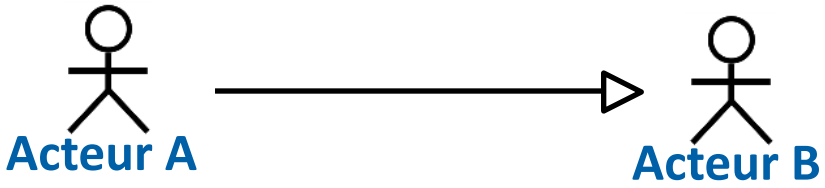


- ***Remarque :*** la **généralisation** se traduit par le concept d'**héritage**.

Relations dans un diagramme de CU

Relation *Acteur-Acteur*

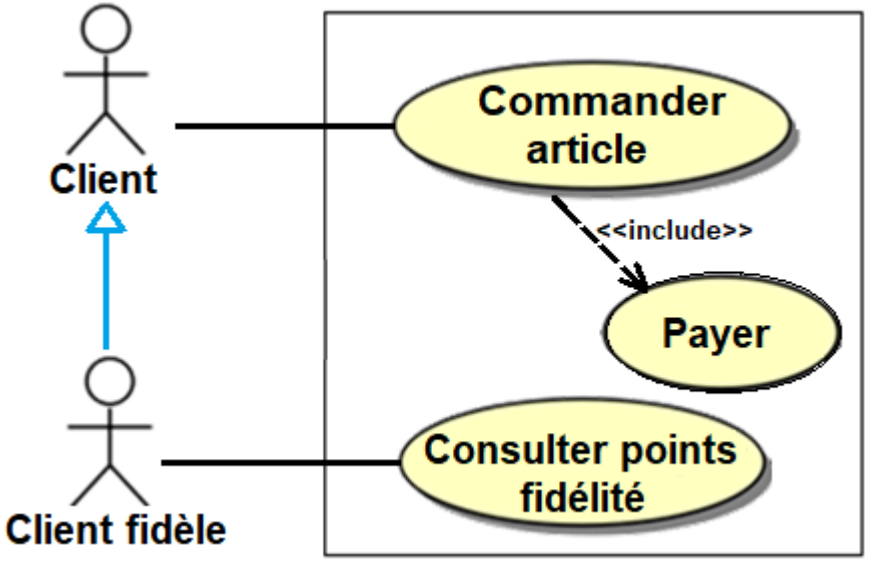
- La seule **relation** possible **entre acteurs** est la **généralisation** (—▷).



- L'**acteur B** est une **généralisation** de l'**acteur A** ssi tous les **CU** de **B** sont accessible par **A**, mais pas l'inverse.

- ***Exemple***
  DCU d'un site de vente en ligne :

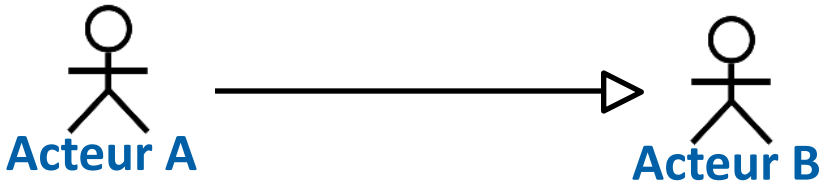
Un **Client fidèle** est une sorte de **client**. Il peut faire tout ce que peut faire un **client**, plus d'autres choses



Relations dans un diagramme de CU

Relation *Acteur-Acteur*

- La seule **relation** possible **entre acteurs** est la **généralisation** (—▷).

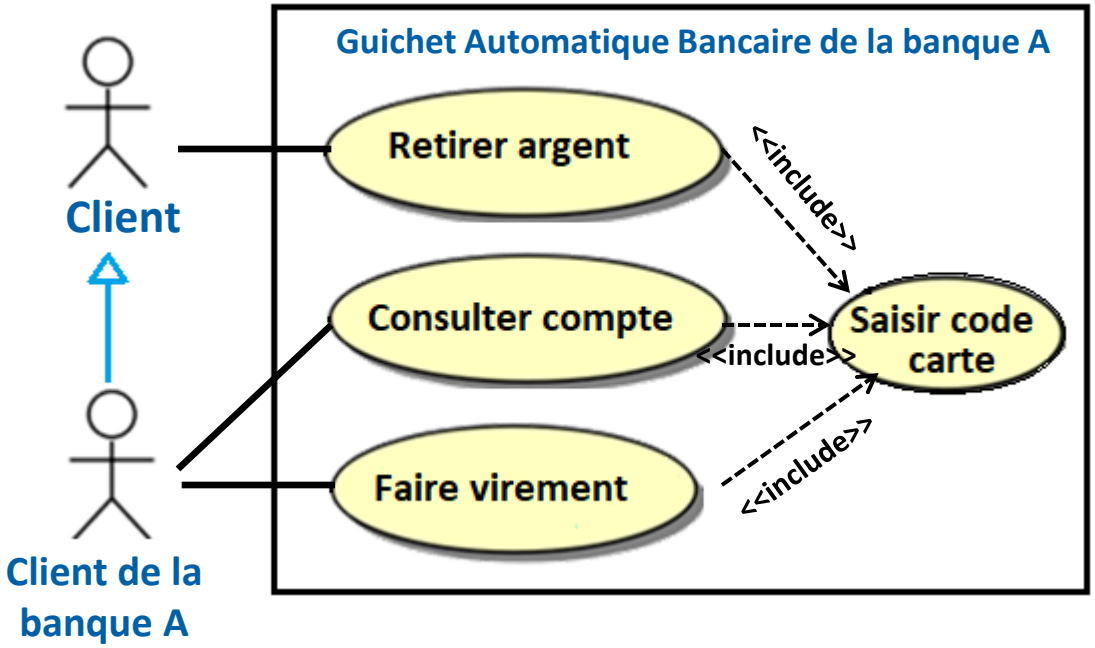


- L'**acteur B** est une **généralisation** de l'**acteur A** ssi tous les **CU** de **B** sont accessible par **A**, mais pas l'inverse.

- ***Exemple***

DCU de gestion de banques:

Un **Client de la banque A** est une sorte de **client**. Il peut faire tout ce que peut faire un **Client**, plus d'autres choses

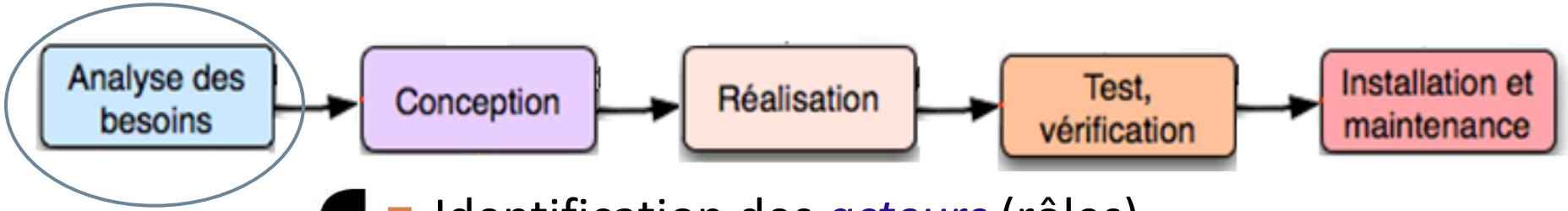


Modélisation des besoins avec UML

Modélisation des besoins avec UML

Etapes de la modélisation des besoins

- Dans le **cycle de vie d'un logiciel**, les principales **étapes** de la **modélisation fonctionnelle** sont :



- Identification des *acteurs* (rôles)
- Identification des *CU* (par acteur)
- Réalisation des *diagrammes de CU*
- *Description textuelle des CU* (scénarios)
- *Organisation des CU* (relation entre CU + packages)

- En plus de **descriptions textuelles**, le scénario peut être représenté en utilisant un **diagramme d'activité.**

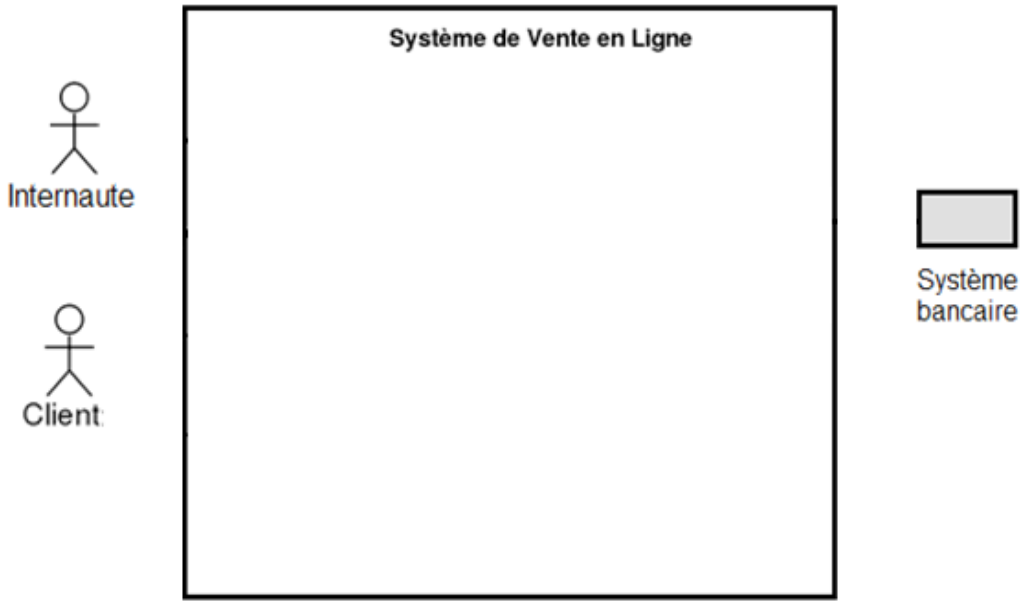
Modélisation des besoins avec UML

Identification des acteurs

- Pour trouver les **acteurs** d'un système, il faut identifier quels sont les **rôles** que vont devoir jouer ses **utilisateurs** (client, administrateur, ….).

- En plus des **utilisateurs**, les **acteurs** peuvent être d'**autres systèmes** externes (systèmes informatiques, des périphériques ou des logiciels, …)

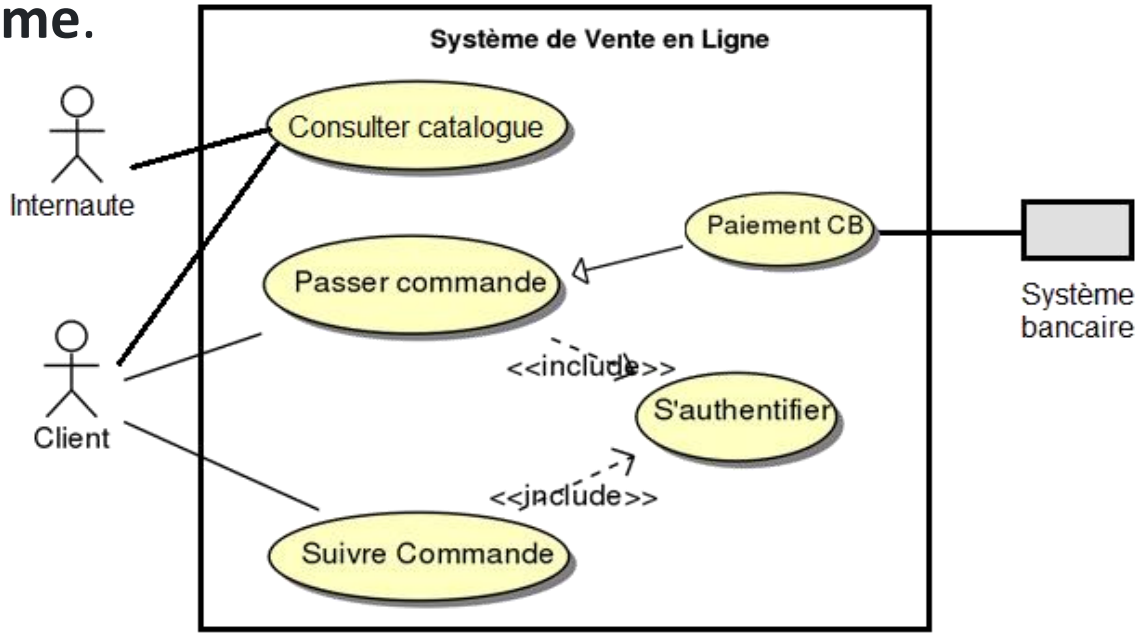
- Pour faciliter la recherche des **acteurs**, on peut imaginer les **frontières** du système :
  - tout ce qui est à l'**extérieur** et qui interagit avec le système est un **acteur**.
  - tout ce qui est à l'**intérieur** est une **fonctionnalité**.



Modélisation des besoins avec UML

Identification des CU

- Pour identifier les **CU**, il faut se placer du point de vue de chaque **acteur** et déterminer **comment** et **pourquoi** il **se sert du système**.
- L'**ensemble** des **CU** doit décrire **exhaustivement** les **exigences fonctionnelles** du **système**.

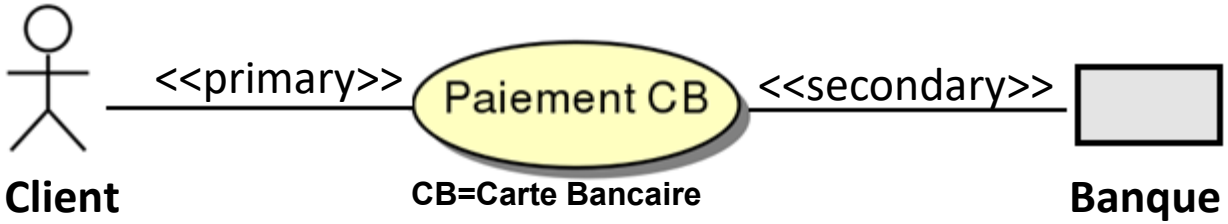


- Il ne faut pas faire apparaître les **détails** des **CU**, mais il faut rester au niveau des **grandes fonctions** du système.

Modélisation des besoins avec UML

Description textuelle des CU

- Le **diagramme de CU** décrit les **fonctions** d'un système, mais ne détaille le **dialogue** entre les **acteurs** et les **CU**.



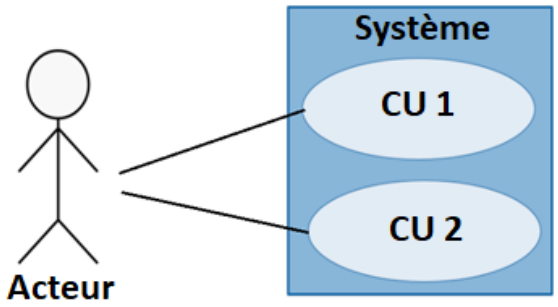
Un simple nom est tout à fait insuffisant pour décrire un cas d'utilisation !!

Il est recommandé de rédiger une description textuelle des CU pour qu'il n'y ait aucune ambiguïté concernant son déroulement.

Modélisation des besoins avec UML

Description textuelle des CU

- La **description textuelle** d'un **CU** n'est pas normalisée (il n'y a aucune proposition de UML sur ce point).

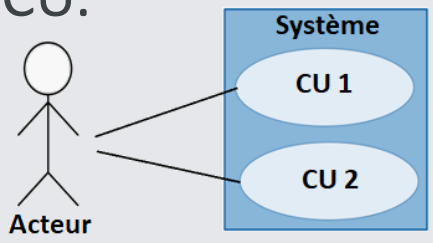


- La **description textuelle** couramment utilisée se compose de 3 parties :
  1. **Identification du CU** : contient les informations concernant le CU.
  2. **Scénario** : décrit le fonctionnement du **CU** sous la forme d'une séquence de **messages** échangés entre les **acteurs** et le **système**.
  3. **Exigences non fonctionnelles** (optionnelle) : décrit les besoins non fonctionnelles du système (fiabilité, type IHM,...).

Modélisation des besoins avec UML

Description textuelle des CU : *Identification*

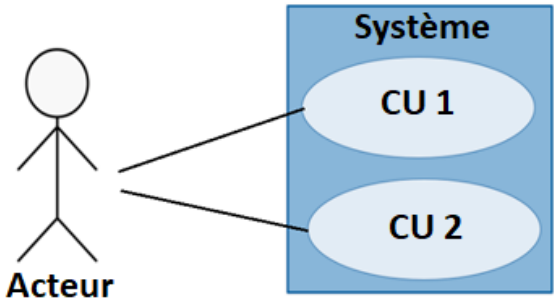
- **1. Identification** : Un CU est **identifié** par un ensemble de données :
  - **Nom :** utiliser une tournure à l'infinitif (ex.:passer commande).
  - **Objectif :** une description résumée permettant de comprendre l'intention principale du CU.
  - **Acteurs principaux :** ceux qui vont réaliser le CU.
  - **Acteurs secondaires :** ceux qui ne font que recevoir des informations à l'issue de la réalisation du CU.
  - **Dates :** date de création/mise à jour du CU.
  - **Responsable :** nom des responsables.
  - **Version :** numéro de version.



Modélisation des besoins avec UML

Description textuelle des CU : *Scénario*

- **2/Description des scénarios** : Le fonctionnement du CU est **décrit** sous la forme d'une séquence de **messages** échangés entre les **acteurs** et le **système** en précisant :



1. **Les préconditions :** décrivent l'**état du système** avant l'exécution du **CU**.

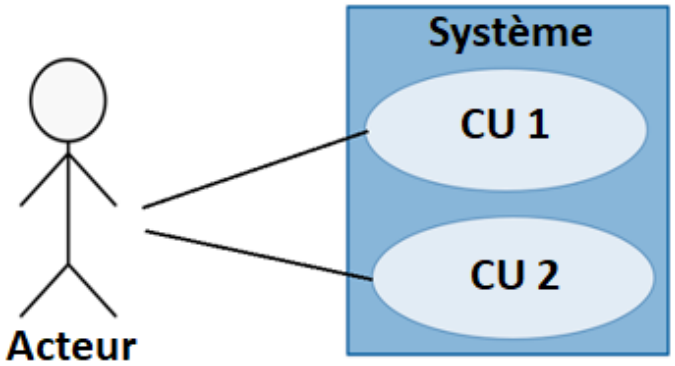
1. **Des scénarios** : décrivent les échanges d'événements entre l'**acteur** et le **système**. Il y a plusieurs types de **scénarios** :
   - le **scénario nominal** : correspond à un déroulement **normal** du **CU**.
   - des **scénarios alternatifs** : ce sont les **variantes** du scénario nominal.
   - les **scénarios d'exception** : décrivent ce qui se passe lors d'une **erreur**.

2. **Des postconditions :** décrivent l'**état du système** à l'issue des différents scénarios.

Modélisation des besoins avec UML

Description textuelle des CU : *Exigences non fonctionnelles*

- **3/Exigences non fonctionnelles** (optionnelle) : décrit tous ce qui **ne peut pas se présenter** comme **fonction** utilisateur visible :
  - Performance
  - Fiabilité et efficacité
  - Portabilité
  - Maintenabilité
  - Scalabilité (évolutivité)
  - Sécurité
  - Spécifications techniques
  - Besoins en termes d'interface graphique
  - ….



Modélisation des besoins avec UML

Description textuelle des CU : *Exemple*

Soit le cas d'utilisation suivant :



- **I. Identification :**

- **Titre** : retirer de l'argent
- **Résumé** : CU qui permet à un client de la banque de retirer de l'argent
- **Acteur principal** : Client (porteur de la carte bancaire)
- **Acteur secondaire** :/ Le Système d'Information de la banque.
- **Date** : 28/08/2021
- **Responsable** : A.
- **Version** : 1.0



Modélisation des besoins avec UML

Description textuelle des CU : *Exemple*



- **II. 1. Scénario nominal :**

1) Le Porteur de carte introduit sa carte dans le DAB.
2) Le DAB vérifie que la carte introduite est bien une carte bancaire.
3) Le DAB demande le code de la carte au Porteur de carte.
4) Le Porteur de carte saisit son code.
5) Le DAB compare ce code avec celui qui est codé sur la carte.
6) Le DAB demande le montant désiré au Porteur de carte.
7) Le Porteur de carte saisit le montant.
8) Le DAB vérifie si le montant demandé est inférieur ou égale au crédit hebdomadaire.
9) Le DAB rend la carte et demande au Porteur de carte de la retirer.
10) Le Porteur de carte reprend sa carte.
11) Le DAB demande au Porteur de carte s'il désire un ticket.
12) Le Porteur de carte accepte le ticket.
13) Le DAB délivre le ticket et les billets. **14)** Le Porteur de carte prend les billets et le ticket.

Modélisation des besoins avec UML

Description textuelle des CU : *Exemple*

- **II.2. Scénarios alternatifs :**



- **Scénario alternatif SA1:** Le code est erroné pour la 1ère ou la 2ème fois.
  - SA1 commence au point 5 du scénario nominale.
  - Le DAB indique que le code est erroné.
  - Le DAB enregistre l'échec.
  - Le scénario reprend au point 3 du scénario nominal.
- **Scénario alternatif SA2:** Le montant demandé est trop élevé.
  - SA2 commence au point 8 du scénario nominal.
  - Le DAB affiche le montant max et demande de ressaisir un montant.
  - Le scénario reprend au point 7 du scénario nominal.
- **Scénario alternatif SA3:** ….

Modélisation des besoins avec UML

Description textuelle des CU : *Exemple*

- **II.3. Scénarios d'erreurs :**



- **Scénario d'exception SE1** : Carte non valide.
  - SE1 commence au point 2 du scénario nominal.
  - Le DAB indique que la carte n'est pas valide, restitue la carte et met fin au **CU**.
- **Scénario d'exception SE2** : Le code est erroné pour la 3ème fois.
  - SE2 commence au point 5 du scénario nominal.
  - Le DAB indique que le code est erroné pour la 3ème fois, confisque la carte et met fin au **CU**.
- **Scénario d'exception SE3 :** ...

Modélisation des besoins avec UML

Description textuelle des CU : *Exemple*

- **III. Exigences non fonctionnelles :**



- **Sécurité** : La saisie du code confidentiel ne doit pas faire apparaître le code à l'écran.



- **Ergonomie** : L'interface doit être simple, intuitive et cohérente.



- **Performance** : Garantir des temps de réponse rapides et un service fluide.

- **Utilisabilité** : Le nombre d'étapes pour accomplir une tâche doit être réduit au minimum.

Diagramme de package (paquetages)

- Notion de package
- Diagramme de packages
- Dépendances entre packages

Package en UML

Notion de package

- Un système de grande taille peut être décomposé en plusieurs parties appelées **packages (paquetages)**.

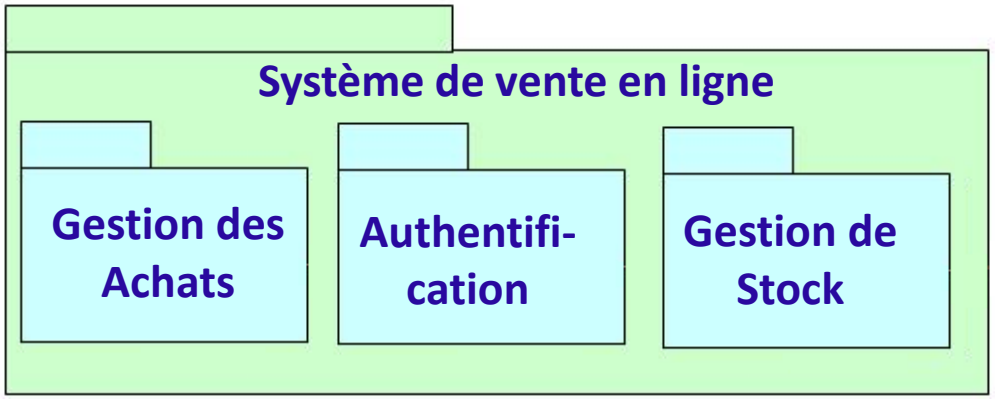
- Un **package** est un regroupement de différents éléments d'un système (classes, diagrammes, fonctions, sous-packages,…).

- Il est représenté par un **dossier** avec son nom à l'intérieur :



- ***Exemple*** :

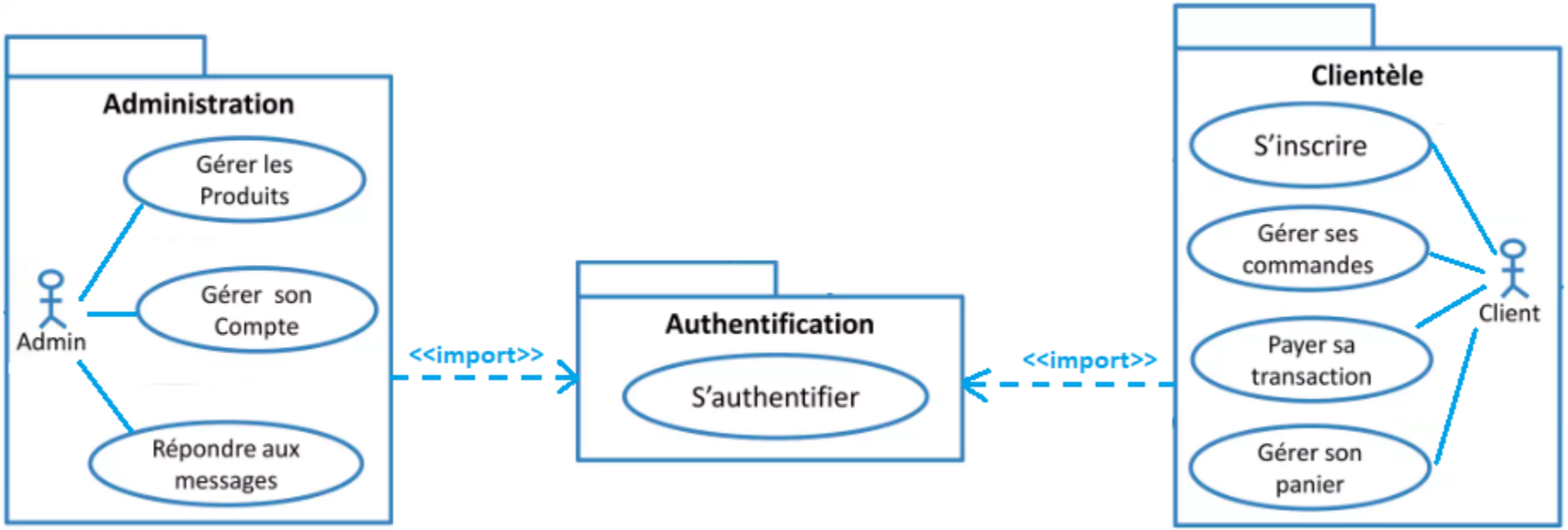
Un système de vente en ligne peut être décomposé en packages :



Package en UML
Utilisation

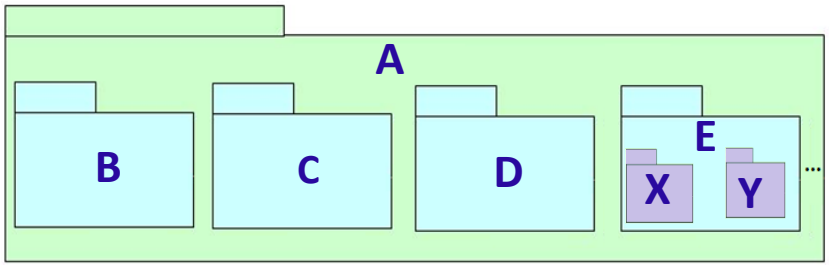
- Un **package** regroupe des éléments du **modèle** de tout type, y compris d'autres **packages**.
- On peut utiliser les **packages** dans plusieurs types de diagrammes : diagrammes de CU, diagrammes de classes, …

- ***Exemple*** : Paquetage des CU d'un système bancaire :



Package en UML
Utilisation

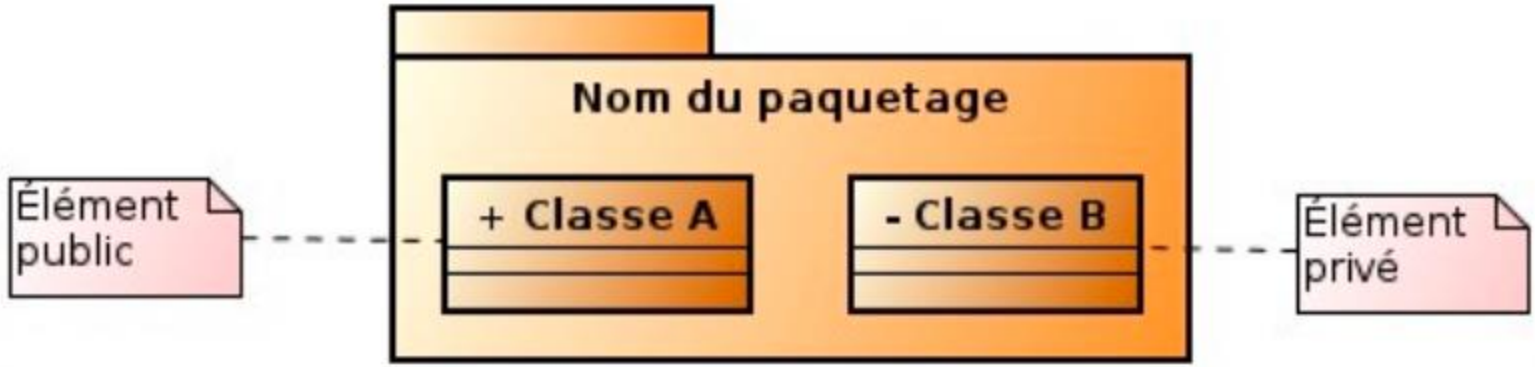
- Pour faire appel à un élément d'un package, on indique le nom du package suivi de deux fois deux points (**::**) puis du nom de l'élément.
- ***Exemples*** : soit le diagramme de package suivant :



  - faire appel au package **D** : on écrit : **A::D**
  - faire appel package **Y** : on écrit : **A::E::Y**

Package en UML
Visibilité

- Chaque éléments d'un package est soit :
  - **Privé** (désigné par -) : invisible à l'extérieur du package.
  - **Public** (désigné +) : visible et accessible de l'extérieur du package.
- ***Exemple*** :

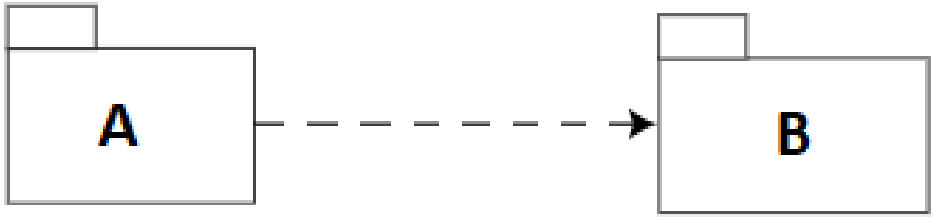


- **NB** : Par défaut, les éléments d'un paquetage sont publics.

Diagramme de package

Présentation

- Le **diagramme de package** est utilisé pour représenter les **packages** composant un système, ainsi que les **relations** qui lient ces packages.
- **Objectif** : simplifier les diagrammes complexes sous une forme visuelle bien ordonnée.

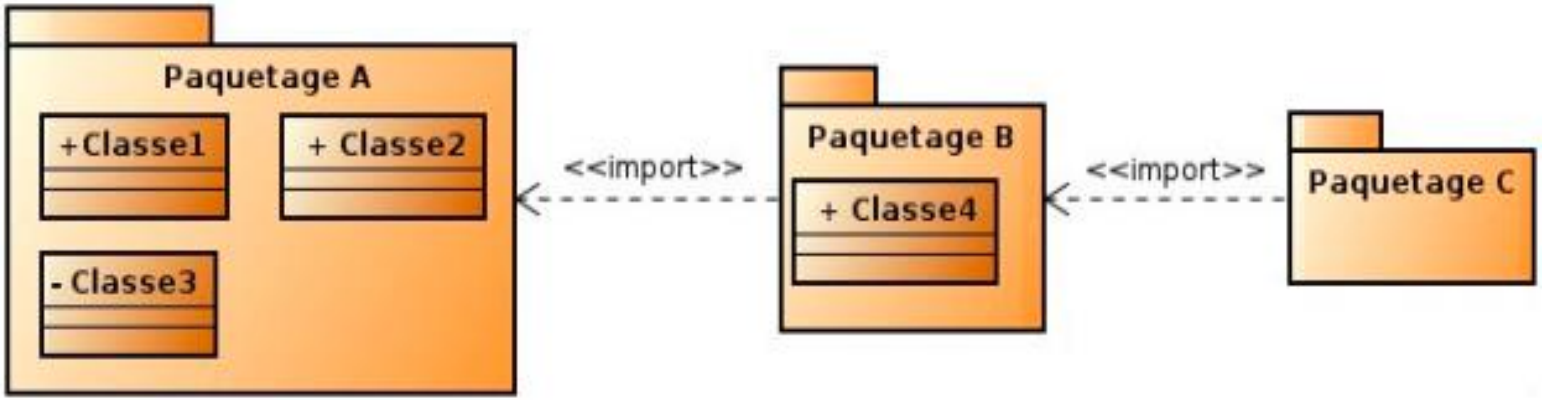


- Les **dépendances** entre packages sont des représentation visuelle de la façon dont un élément (ou un ensemble d'éléments) dépend d'un autre ou l'influence.
- Il existe plusieurs types de **dépendances** :
  - dépendances d'accès ;
  - dépendances d'importation ;
  - ...

Diagramme de package

Dépendance de type « *import* » :

- Elle correspond à l'**importation** par un package **B** de tous les **éléments publics** d'un package **A**. Les éléments importés :
  - auront la visibilité « **public** » dans le package B (et seraient donc aussi transmis à un paquetage C qui ferait une importation du package B).
  - seront accessibles au package B sans avoir à utiliser explicitement le nom du package A.
- ***Exemple*** :

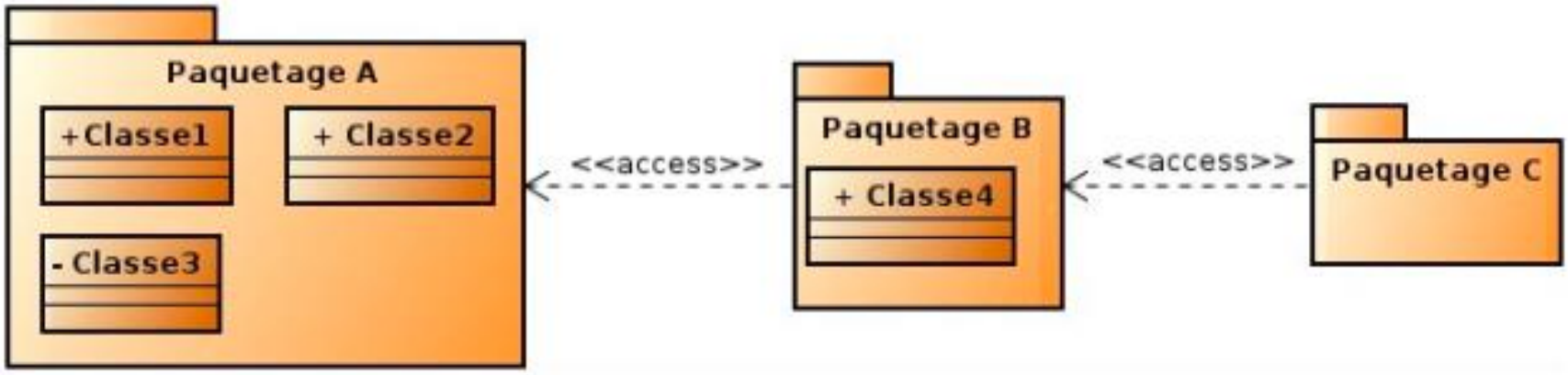


- La dépendance de type « import » est représentée par une flèche pointillée muni du stéréotype << ***import*** >>.

Diagramme de package

Dépendance de type « *access* » :

- Elle correspond à l'**accès** par un package B de tous les **éléments publics** d'un package A. Ces éléments :
  - auront la visibilité **privé** dans le package B
  - ils ne peuvent donc pas être transmis à un paquetage C qui ferait une importation ou un accès au paquetage B.
- ***Exemple*** :

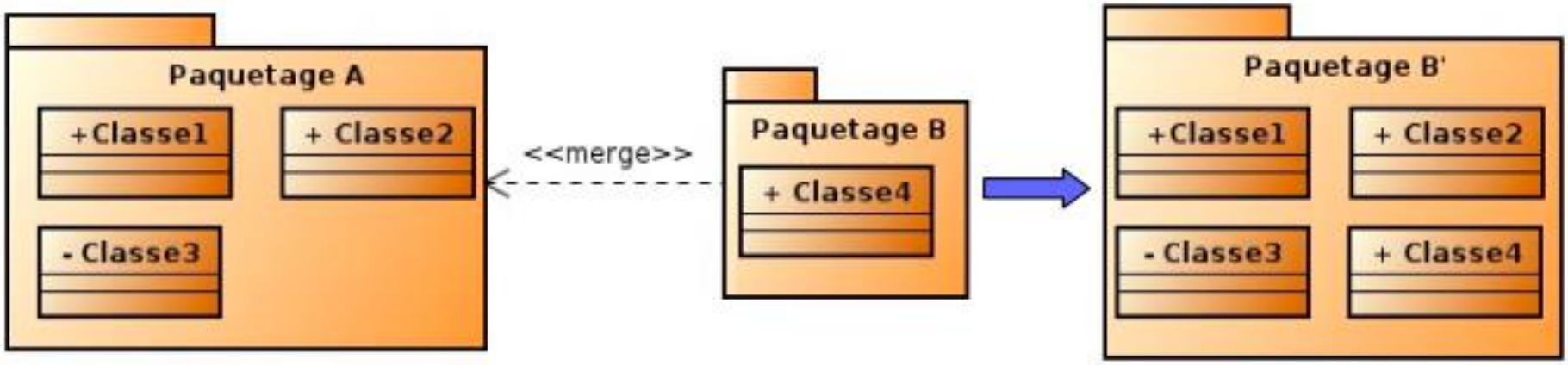


- La dépendance de type « access » est représentée par une flèche pointillée muni du stéréotype << ***acces*** >>.

Diagramme de package

Dépendance de type « *merge* » :

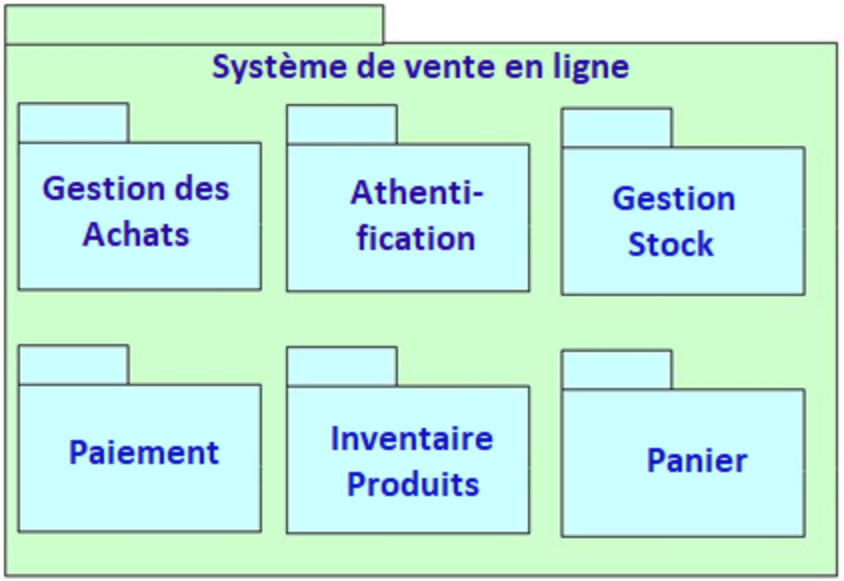
- Elle correspond à la fusion de 2 packages en un seul.
- La dépendance de type « merge » est représentée par une flèche pointillée muni du stéréotype << ***merge*** >>.
- ***Exemple*** :



➔Le package A est fusionné dans le package B (A n'est pas modifié alors que B est écrasé pour accueillir la fusion des 2 paquetages).

Diagramme de package

Exemple :



- On peut décomposer le **diagramme de CU** d'un « système de vente en ligne » en **packages**.

- Diagramme de CU du package « gestion des stocks » :

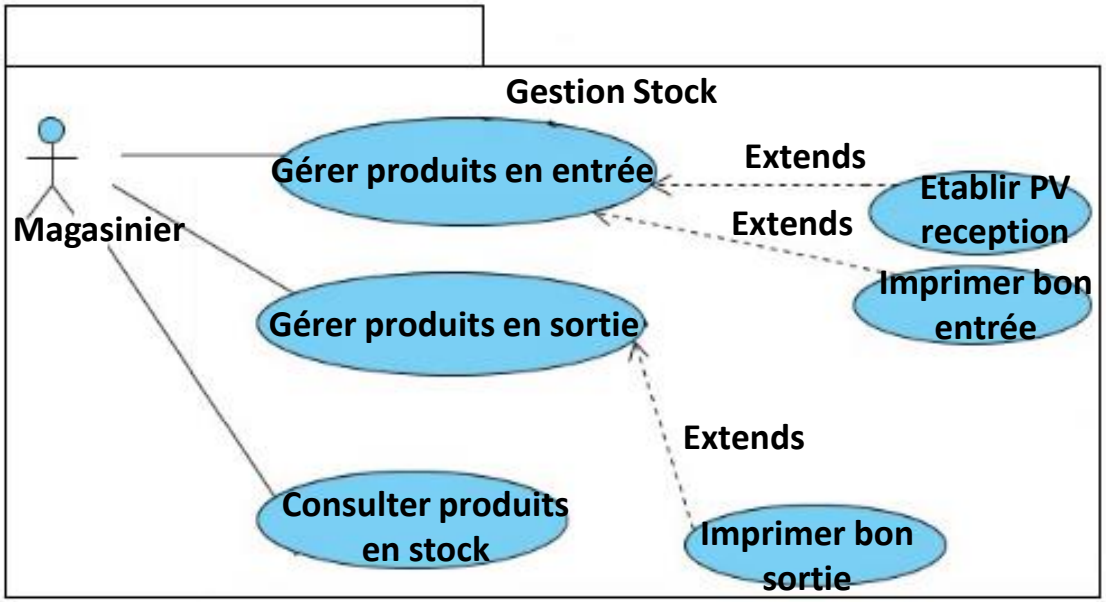
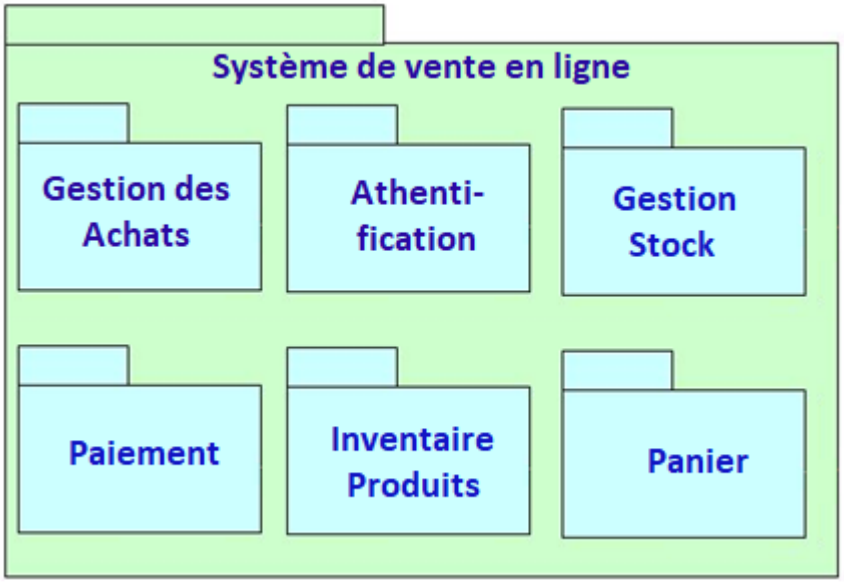


Diagramme de package

Exemple :



- On peut définir des relations de dépendances entre les packages.
- **Diagramme de package :**

